

Chat Transcript

Complete transcript of this conversation segment, in order.

User

My loop skips reactions missing from activity_data, but glucose exchange needs a separate bound change. Is the order of my conditions correct? How should I fix it?

```
for reaction in model.reactions:
    # nd
    if reaction.id not in activity_data:
        continue

    # glucose exchange reaction - use high absolute default bound
    if reaction.id == "EX_glc__D_e":
        reaction.lower_bound = -1000.0
        reaction.upper_bound = 1000.0
        continue

    # APTM case - leave lower flux as it is, set upper bound to activity data
    if reaction.id == "ATPM":
        reaction.upper_bound = activity_data[reaction.id]
        continue

    # two first cases
    if reaction.reversibility:
        reaction.upper_bound = activity_data[reaction.id]
        reaction.lower_bound = -activity_data[reaction.id]
    else:
        reaction.upper_bound = activity_data[reaction.id]
        reaction.lower_bound = 0
```

Assistant

Yes — the order is the problem.

Right now, this condition runs first:

```
if reaction.id not in activity_data:
    continue
```

So if "EX_glc__D_e" is not in activity_data, Python immediately executes continue and never reaches your special glucose case.

You should handle reactions that need special treatment before checking whether they exist in activity_data:

```
for reaction in model.reactions:

    # Glucose exchange: special case, does not need activity_data
    if reaction.id == "EX_glc__D_e":
        reaction.lower_bound = -1000.0
        reaction.upper_bound = 1000.0
        continue

    # Skip reactions without activity data
    if reaction.id not in activity_data:
        continue

    # ATPM: keep lower bound unchanged, only change upper bound
    if reaction.id == "ATPM":
        reaction.upper_bound = activity_data[reaction.id]
        continue

    # Normal reactions
    if reaction.reversibility:
        reaction.upper_bound = activity_data[reaction.id]
        reaction.lower_bound = -activity_data[reaction.id]
```

```

else:
    reaction.upper_bound = activity_data[reaction.id]
    reaction.lower_bound = 0

```

The important idea is that the loop works top → bottom. As soon as Python encounters `continue`, it skips everything below it and moves to the next reaction.

So your logic becomes:

glucose special case → check activity data exists → ATPM special case → normal reversible/irreversible case.

For "EX_glc__D_e", it now doesn't matter whether it's present in `activity_data`: it will always get [-1000, 1000].

User

should I just move this in the beginning? if `reaction.id` not in `activity_data`: `continue`

Assistant

Not quite — move the glucose condition before it, rather than moving if `reaction.id` not in `activity_data` to the beginning (it is already at the beginning).

Like this:

```

for reaction in model.reactions:

    # Glucose must be checked FIRST
    if reaction.id == "EX_glc__D_e":
        reaction.lower_bound = -1000.0
        reaction.upper_bound = 1000.0
        continue

    # THEN skip reactions without activity data
    if reaction.id not in activity_data:
        continue

    # ATPM
    if reaction.id == "ATPM":
        reaction.upper_bound = activity_data[reaction.id]
        continue

    if reaction.reversibility:
        reaction.upper_bound = activity_data[reaction.id]
        reaction.lower_bound = -activity_data[reaction.id]
    else:
        reaction.upper_bound = activity_data[reaction.id]
        reaction.lower_bound = 0

```

So yes: just swap the first two blocks.

The reason is simple: if glucose isn't in `activity_data`, your current first `continue` says "skip this reaction entirely", so the glucose-specific code is never reached.

User

why does setting `EX_glc__D_e.lower_bound = -5`` limit glucose uptake to 5 mmol/gDW/h?

Assistant

Because in COBRA/FBA models, exchange reactions use a sign convention:

- Negative flux = uptake into the cell
- Positive flux = secretion out of the cell

So for the glucose exchange reaction `EX_glc__D_e`, setting:

```

model.reactions.EX_glc__D_e.lower_bound = -5

```

means its flux must satisfy:

$$v_{\text{glucose}} \geq -5$$

If, for example, the upper bound is 1000, then:

$$-5 \leq v_{\text{glucose}} \leq 1000$$

The most negative glucose flux allowed is therefore -5 mmol/gDW/h. Since negative flux represents uptake, this corresponds to a maximum uptake rate of 5 mmol/gDW/h.

Think of the sign like this

Extracellular glucose
↓ uptake
CELL

flux = -5 → uptake of 5
flux = -3 → uptake of 3
flux = 0 → no exchange
flux = +3 → secretion of 3

So when someone says "limit glucose uptake to 5", you write:

```
EX_glc__D_e.lower_bound = -5
```

The slightly confusing part is that more glucose uptake means a more negative number. Therefore, -5 is an uptake limit: the optimizer cannot choose -6, -10, etc.

User

My growth rates are 0.8733 and 0.4156 h⁻¹ before and after limiting glucose uptake, is it normal?

Assistant

Yes — that result is completely plausible in FBA.

You changed the maximum glucose uptake to 5 mmol/gDW/h, and your growth rate changed from:

$$\mu_{\text{before}} = 0.8733 \text{ h}^{-1}$$

to

$$\mu_{\text{after}} = 0.4156 \text{ h}^{-1}$$

That's about a 52.4% decrease in predicted growth.

Biologically, this makes sense: glucose is a major carbon and energy source. When you restrict how much glucose the model can import, less carbon is available for producing ATP and biomass precursors, so the maximum achievable biomass flux decreases.

One useful check is:

```
solution = model.optimize()

print("Growth:", solution.objective_value)
print("Glucose flux:", solution.fluxes["EX_glc__D_e"])
```

If after setting:

```
model.reactions.EX_glc__D_e.lower_bound = -5
```

you get something close to:

```
Growth:      0.4156
Glucose flux: -5.0
```

that's especially informative. It means the optimizer is pushing glucose uptake right against the new limit, so glucose availability is actually constraining growth.

If instead you got, say, -2.3, then the -5 bound wouldn't be active, and you'd want to investigate what else caused the growth change.

User

could calling `model.optimize()` change the results stored in `solution_a`?

Assistant

No. Calling `model.optimize()` again does not normally change an already stored `solution_a`.

For example:

```
solution_a = model.optimize()

model.reactions.EX_glc__D_e.lower_bound = -5

solution_b = model.optimize()
```

Here:

- `solution_a` contains the solution from before changing the glucose bound.
- `solution_b` contains the solution from after changing it.
- Changing the model and optimizing again does not overwrite `solution_a`.

So this is perfectly fine:

```
print(solution_a.objective_value) # e.g. 0.8733
print(solution_b.objective_value) # e.g. 0.4156
```

Important distinction

`model.optimize()` calculates a new solution based on the model's current state.

So:

```
solution_a = model.optimize() # glucose unlimited
```

stores the first result.

Then:

```
model.reactions.EX_glc__D_e.lower_bound = -5
solution_b = model.optimize() # glucose limited
```

creates a new `Solution` object.

You can even verify it:

```
print(solution_a is solution_b)
```

which should give:

```
False
```

One thing to watch out for is accidentally doing:

```
solution_a = model.optimize()

# later...
solution_a = model.optimize()
```

because then you explicitly overwrite the variable `solution_a` with the new solution.

User

create a downloadable PDF of this entire chat, including all my questions, your answers, and code in order, label the speakers and don't summarize or omit anything